ARCOR – DESAFIO DAS MARCAS

Jogo da Tartaruga: aventura entre ilhas com quiz

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

VICTOR HUGO DE MELO RIBEIRO - RA: 26029726

GILDO SOUZA JUNIOR - RA: 26029658

KAINAN DE SOUZA ALVES - RA: 26029656

RAISSA SOUSA NUNES - RA: 26029670

KAUÊ LIMA CERQUEIRA - RA: 26029835

São Paulo
2026

Sumário

1. INTRODUÇÃO 3
1.1 Processo de desenvolvimento de Software 3
1.2 Apresentação 3
2. DESIGN SPRINT – Ideação e prototipação do desafio 3
2.1 Desafio 3
2.2 Entender Mapear 4
2.2.1 Entendimento e observação. COMO PODEMOS? 4
2.2.2 Pesquisa Desk 4
2.3 Ideação – desenho da solução (trilha do usuário) 4
2.4 Prototipagem 5
3. REQUISITOS DE SISTEMA 5
3.1 REQUISITOS FUNCIONAIS DE SOFTWARE 5
3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE 8
4. DIAGRAMAS DE UML 12
4.1 CASOS DE USO 12
4.2 DIAGRAMA DE CLASSE 15
5. ESPECIFICAÇÃO DA ESTRUTURA DE UMA ROTINA DO SISTEMA 16
5.1 Especificação da rotina 16
5.2 Codificação da rotina 17
6. ARQUITETURA DO SISTEMA 17
6.1 Visão Geral do Sistema 17
6.2 Arquitetura Geral 18
6.3 Componentes Arquiteturais e Camadas 18
6.3.1 Camada de Apresentação (Frontend) 18
6.3.2 Camada de Aplicação (Backend/API) 18
6.3.3 Camada de Domínio (Negócio) 18
6.3.4 Camada de Persistência (Banco de Dados) 18
6.4 Diagrama da Arquitetura 18
6.5 Organização de Pastas e Módulos 20
6.6 Integrações Externas 20
6.7 Ambiente e Implantação 20
8. REFERÊNCIAS BIBLIOGRÁFICAS 20

1. INTRODUÇÃO

1.1 Processo de desenvolvimento de Software

O desenvolvimento do sistema segue o modelo em cascata com realimentação: as etapas de Análise, Projeto, Codificação, Testes e Implementação são executadas em sequência e, quando um problema é encontrado, a equipe pode retornar a uma etapa anterior para corrigi-lo, conforme o diagrama abaixo.

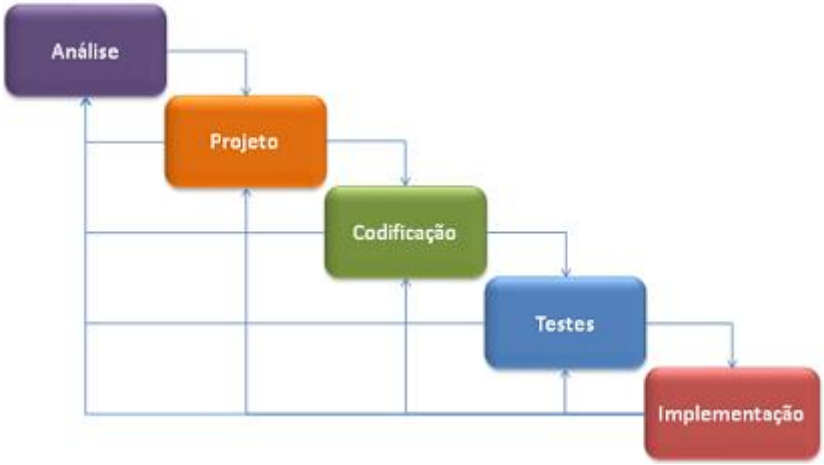


Figura 1: Processo de desenvolvimento de software (modelo em cascata).

1.2 Apresentação

A Arcor é uma empresa do ramo alimentício, com um portfólio que reúne chocolates, biscoitos, guloseimas e outras categorias. Como ação de marketing e relacionamento com o público, a empresa propôs às equipes do 1º semestre do curso de Análise e Desenvolvimento de Sistemas da FECAP o desenvolvimento de um sistema digital lúdico e educativo, capaz de aproximar o jogador da marca Arcor e de suas marcas, linhas e produtos, como Tortuguita e Triunfo.

Neste Projeto Interdisciplinar, a equipe desenvolve o ARCOR – Desafio das Marcas, também chamado de Jogo da Tartaruga: uma aventura em que uma tartaruga viaja de ilha em ilha. Cada ilha é temática de uma marca e possui um nível de dificuldade (fácil, média ou difícil). Em cada ilha, a tartaruga enfrenta inimigos e, para avançar, o jogador precisa responder corretamente às perguntas de um quiz sobre as marcas e os produtos da Arcor.

Todo o conteúdo do jogo (marcas, produtos, ilhas, perguntas, alternativas e parâmetros de pontuação) fica armazenado em um banco de dados relacional MySQL, e não no código do programa. Assim, novos conteúdos podem ser incluídos sem alterar a lógica do jogo. O público-alvo definido pela equipe é o público juvenil.

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Desenvolver um sistema digital lúdico e educativo que utilize perguntas e regras de jogo armazenadas em Banco de Dados para promover a familiarização do usuário com a Arcor, suas marcas, linhas e produtos, integrando conhecimentos das Unidades Curriculares do 1º Semestre de Análise e Desenvolvimento de Sistemas.

2.2 Entender Mapear

2.2.1 Entendimento e observação. COMO PODEMOS?

- Público juvenil. Definido
- Movimentação scriptada. Definido
- Ilhas temáticas das marcas. Definido

- Como podemos definir o sistema de pontuação do jogo?
- Como podemos definir a ordem e o sistema de dificuldade?
- Como podemos definir o sistema de bonificação?
- Como podemos refinar o gráfico no pós?
- Como podemos prender a atenção do jogador?

2.2.2 Pesquisa Desk

Arte similar à de Undertale.

2.3 Ideação – desenho da solução (trilha do usuário)

- Se tivesse superpoderes? Poder de dominar o tempo e criar um jogo mais qualificado.
- Se eu tivesse muito dinheiro? Contrataria um estúdio consolidado para fazer o jogo/quiz.
- Se eu não tivesse dinheiro nenhum? Jogo de tabuleiro/manual.
- Se fosse o Dick Vigarista e quisesse boicotar? Hackeava o jogo e pulava etapas.

Crazy 8 ideias: Mundo aberto

1. Tartaruga matando os boss com foice
2. Chocolate faz mal pra saúde
3. Tartaruga matando os plásticos
4. Top nacional ganhar a empresa inteira
5. Propaganda infantil incentivando consumo de chocolate
6. Tortuguita pisando nos concorrentes no final do jogo
7. Tortuguita lutando pelos seus direitos
8. Arena de guerra das tortuguitas



Figura 2: Registro da ideação (tortuguitas de chocolate).

2.4 Prototipagem

[A EQUIPE PREENCHE: inserir aqui as imagens do protótipo (telas do jogo, esboços das ilhas e da tartaruga) e uma breve descrição de cada uma.]

3. REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

RFS01	
Função	Cadastrar usuário
Descrição	Permite que um novo jogador crie uma conta no site para poder jogar e ter seu progresso registrado.
Entradas	Nome, email, senha e confirmação de senha.
Fonte	Jogador, por meio do formulário de cadastro.
Saídas	Mensagem de cadastro realizado e redirecionamento para a tela de login, ou mensagem de erro indicando o problema.
Ação	1. O jogador acessa a tela de cadastro e preenche os campos. 2. O sistema verifica se todos os campos foram preenchidos, se o email tem formato válido e se a senha e a confirmação são iguais. 3. O sistema verifica se o email já está cadastrado. 4. O sistema grava o usuário, com a senha criptografada e a data de cadastro. 5. O sistema exibe a confirmação e leva o jogador à tela de login.

RFS02	
Função	Autenticar usuário
Descrição	Permite que o jogador cadastrado entre no sistema e saia dele, garantindo que cada pessoa acesse apenas as próprias partidas.
Entradas	Email e senha.
Fonte	Jogador, por meio do formulário de login. Dados do usuário armazenados no banco de dados.
Saídas	Acesso à tela inicial do jogo, ou mensagem de email ou senha inválidos.
Ação	1. O jogador informa email e senha na tela de login. 2. O sistema busca o usuário pelo email e compara a senha informada com a senha armazenada. 3. Se os dados estiverem corretos, o sistema inicia a sessão do usuário e exibe a tela inicial. 4. Se estiverem incorretos, o sistema exibe a mensagem de erro e mantém o jogador na tela de login. 5. Ao escolher sair, o sistema encerra a sessão e retorna à tela de login.

RFS03	
Função	Iniciar partida
Descrição	Permite que o jogador autenticado comece uma nova partida do jogo ou continue uma partida em andamento.
Entradas	Comando “Novo Jogo” ou “Continuar”.
Fonte	Jogador autenticado. Dados da partida e da primeira ilha armazenados no banco de dados.
Saídas	Partida criada ou retomada, com a tartaruga posicionada na ilha atual, exibindo vida e pontuação.
Ação	1. O sistema confirma que o usuário está autenticado. 2. Se houver partida em andamento, o sistema oferece continuar ou começar outra. 3. Em um novo jogo, o sistema cria a partida com data de início, status “Em andamento”, vida atual inicial e pontuação zero. 4. O sistema associa a partida à primeira ilha (menor ordem). 5. O sistema exibe a ilha, a tartaruga, a vida e a pontuação.

RFS04	
Função	Percorrer as ilhas
Descrição	Controla a viagem da tartaruga de ilha em ilha, respeitando a ordem e a dificuldade (fácil, média ou difícil). Uma ilha só é liberada quando o jogador atinge os acertos necessários da ilha anterior.
Entradas	Escolha do jogador de avançar para a próxima ilha.
Fonte	Jogador. Dados das ilhas (descrição, dificuldade, ordem e acertos necessários) e progresso da partida no banco de dados.
Saídas	Ilha exibida com descrição e dificuldade, aviso de ilha bloqueada ou mensagem de fim de jogo na última ilha.
Ação	1. O jogador solicita avançar para a próxima ilha. 2. O sistema verifica se os acertos necessários da ilha atual foram atingidos. 3. Se sim, o sistema registra a passagem da partida pela nova ilha e exibe sua descrição e dificuldade. 4. Se não, o sistema informa que a ilha ainda está bloqueada e quantos acertos faltam. 5. Ao concluir a última ilha, o sistema registra a partida como concluída e exibe a mensagem de fim de jogo.

RFS05	
Função	Enfrentar inimigos
Descrição	Em cada ilha a tartaruga enfrenta inimigos de acordo com a dificuldade. O resultado do combate altera a vida atual e a pontuação da partida.
Entradas	Ação de combate escolhida pelo jogador (por exemplo, atacar).
Fonte	Jogador. Dados da partida (vida atual e pontuação) e da ilha onde a tartaruga está.
Saídas	Resultado do combate, vida atual e pontuação atualizadas e, se a vida chegar a zero, mensagem de derrota.
Ação	1. O sistema apresenta um inimigo compatível com a dificuldade da ilha. 2. O jogador executa a ação de combate. 3. O sistema calcula o dano causado e o dano recebido e atualiza a vida atual da tartaruga. 4. Se o inimigo for derrotado, o sistema soma pontos à partida e libera o quiz da ilha. 5. Se a vida atual chegar a zero, o sistema encerra a partida com status “Derrotada”, registra a data de fim e oferece iniciar um novo jogo.

RFS06	
Função	Responder ao quiz
Descrição	Apresenta as perguntas do quiz da ilha, registra a alternativa escolhida pelo jogador, informa se está correta e atualiza os acertos e a pontuação. Os acertos necessários permitem concluir a ilha.
Entradas	Alternativa escolhida pelo jogador.
Fonte	Jogador. Perguntas e alternativas da ilha armazenadas no banco de dados.
Saídas	Resultado da resposta (correta ou incorreta), explicação da pergunta, acertos e pontuação atualizados e aviso de ilha concluída, quando for o caso.
Ação	1. O sistema seleciona uma pergunta da ilha atual e exibe o enunciado e as alternativas. 2. O jogador escolhe uma alternativa. 3. O sistema registra a escolha vinculada à partida. 4. O sistema verifica se a alternativa escolhida é a correta. 5. O sistema exibe o resultado e a explicação e atualiza os acertos e a pontuação. 6. Quando o número de acertos necessários é atingido, o sistema marca a ilha como concluída e libera a próxima (RFS04).

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

RNF01	
Função	Usabilidade
Descrição	A interface do jogo deve ser fácil de entender, com textos em português e retorno visual claro para ações como acertar, errar e perder vida, de modo que o jogador use o site sem ajuda externa.
Entradas	Interações do jogador com a tela (cliques, toques e teclado).
Fonte	Jogador, usando o site em um navegador.
Saídas	Telas organizadas, mensagens claras e retorno visual imediato para cada ação (acerto, erro, vida e pontuação).
Ação	1. O sistema apresenta menus, botões e textos em português, com nomes claros. 2. A cada ação do jogador, o sistema exibe um retorno visual, como cor, mensagem ou animação simples. 3. Vida, pontuação e ilha atual ficam sempre visíveis durante a partida. 4. Mensagens de erro explicam o problema e indicam como corrigi-lo. 5. Critério de verificação: um usuário novo consegue se cadastrar e iniciar uma partida sem instruções adicionais.

RNF02	
Função	Desempenho
Descrição	O sistema deve responder rapidamente às ações do jogador, para que o jogo seja fluido e o quiz não tenha esperas longas.
Entradas	Solicitações do jogador, como abrir uma tela, carregar uma pergunta e enviar uma resposta.
Fonte	Jogador, por meio do navegador. Dados armazenados no banco de dados.
Saídas	Telas e perguntas carregadas em até 3 segundos e resultado da resposta exibido em até 1 segundo, em conexão comum de internet.
Ação	1. O sistema recebe a solicitação do jogador. 2. O sistema consulta no banco de dados apenas os dados necessários para aquela tela (por exemplo, a pergunta atual e suas alternativas). 3. O sistema processa a solicitação e devolve a resposta ao navegador. 4. Imagens e demais arquivos do jogo são mantidos leves para não atrasar o carregamento. 5. Critério de verificação: os tempos acima são respeitados em testes com uma conexão comum.

RNF03	
Função	Segurança
Descrição	As senhas devem ser armazenadas criptografadas, e cada usuário deve acessar somente as próprias partidas. Os dados enviados devem ser validados para evitar ataques como a injeção de SQL, e a correção das respostas do quiz deve ser feita no servidor.
Entradas	Dados de cadastro e login (email e senha) e alternativas escolhidas no quiz.
Fonte	Jogador, por meio dos formulários e das telas do jogo.
Saídas	Acesso liberado apenas a usuários autenticados, senhas protegidas no banco de dados e respostas verificadas com segurança.
Ação	1. Ao cadastrar um usuário, o sistema transforma a senha em hash antes de gravá-la no banco de dados. 2. O sistema valida e trata todos os dados recebidos antes de usá-los em consultas ao banco de dados. 3. Páginas internas e dados de partidas só são acessados com sessão de usuário válida. 4. O sistema confere se a alternativa escolhida é a correta no servidor, sem enviar essa informação ao navegador antes da resposta. 5. Critério de verificação: nenhuma senha aparece em texto puro no banco de dados e páginas internas não abrem sem login.

RNF04	
Função	Compatibilidade e Portabilidade
Descrição	O site deve funcionar nas versões recentes dos principais navegadores e se adaptar a diferentes tamanhos de tela, como computador, tablet e celular.
Entradas	Acesso do jogador ao site por navegador e dispositivo variados.
Fonte	Jogador, usando computador, tablet ou celular.
Saídas	Jogo exibido e funcionando corretamente, com telas legíveis e utilizáveis em qualquer tamanho de tela.
Ação	1. O sistema usa um layout responsivo, que ajusta tamanhos e posições dos elementos na tela. 2. O sistema usa apenas recursos web com suporte nos navegadores. 3. Textos, botões e alternativas do quiz mantêm tamanho legível em telas pequenas e grandes. 4. Critério de verificação: o jogo abre e funciona sem erros em pelo menos dois navegadores, no computador e no celular.

RNF05	
Função	Confiabilidade
Descrição	O progresso da partida (ilha atual, vida, pontuação e respostas) deve ser salvo automaticamente, e o banco de dados deve manter a consistência entre as informações, sem perguntas sem alternativas ou respostas sem partida.
Entradas	Ações do jogador durante a partida, como responder uma pergunta, vencer um inimigo ou concluir uma ilha.
Fonte	Jogador. Dados da partida armazenados no banco de dados.
Saídas	Progresso salvo e partida retomada do ponto em que parou, com dados consistentes.
Ação	1. A cada resposta, combate ou conclusão de ilha, o sistema grava o progresso no banco de dados. 2. O banco de dados usa chaves primárias e estrangeiras para manter o vínculo entre usuário, partida, ilha, pergunta e alternativa. 3. Se a conexão for interrompida ou o site for fechado, o jogador pode retomar a partida em andamento. 4. Se uma gravação falhar, o sistema informa o erro e não deixa dados incompletos salvos. 5. Critério de verificação: ao fechar e reabrir o site, o jogador retoma a partida perdendo no máximo a última ação.

RNF06	
Função	Manutenibilidade
Descrição	O conteúdo do jogo e o código devem poder ser atualizados com facilidade pela equipe de desenvolvimento.
Entradas	Novos registros de ilhas, perguntas e alternativas, e alterações no código feitas pela equipe de desenvolvimento.
Fonte	Equipe de desenvolvimento e administrador do sistema.
Saídas	Conteúdo do jogo atualizado e funcionando sem necessidade de reescrever o código.
Ação	1. O código é dividido em partes com responsabilidades claras (por exemplo, telas, regras do jogo e acesso ao banco de dados). 2. O código recebe nomes claros e comentários nos trechos principais. 3. Ilhas, perguntas e alternativas ficam em tabelas do banco de dados, e não escritas diretamente no código. 4. Para ampliar o jogo, a equipe apenas insere novos registros nessas tabelas. 5. Critério de verificação: uma nova pergunta ou ilha pode ser incluída apenas inserindo registros no banco de dados.

4. DIAGRAMAS DE UML

4.1 CASOS DE USO

O diagrama a seguir apresenta os três casos de uso principais do sistema. O ator principal é o Jogador. O Banco de Dados MySQL aparece como ator secundário, pois armazena e fornece os dados usados em todos os casos de uso. O caso de uso UC02 inclui o UC01, já que só é possível iniciar uma partida depois de autenticado.

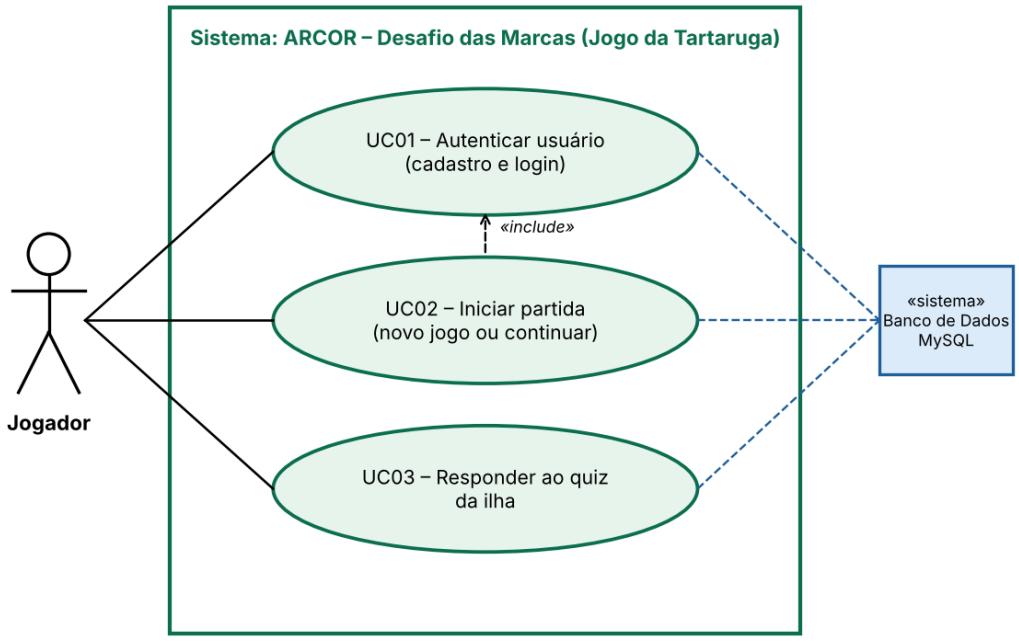


Figura 3: Diagrama de casos de uso do ARCOR – Desafio das Marcas.

UC01 – Autenticar usuário (cadastro e login)	
Ator principal	Jogador.
Ator secundário	Banco de Dados MySQL.
Requisitos relacionados	RFS01, RFS02, RNF03.
Pré-condição	O jogador acessou o site do jogo.
Fluxo principal	1. O jogador escolhe “Entrar”. 2. O sistema exibe o formulário de login. 3. O jogador informa email e senha. 4. O sistema busca o usuário pelo email e compara o hash da senha informada com o armazenado. 5. Se os dados estiverem corretos, o sistema inicia a sessão e exibe a tela inicial do jogo.
Fluxo alternativo A – Novo cadastro	No passo 1, o jogador escolhe “Cadastrar” e informa nome, email, senha e confirmação de senha. O sistema valida os campos, o formato do email, a igualdade das senhas e se o email ainda não existe. O sistema grava o usuário com a senha criptografada e a data de cadastro, exibe a confirmação e leva o jogador ao login.
Fluxo alternativo B – Dados inválidos	Se algum campo estiver vazio ou inválido, se o email já estiver cadastrado ou se email e senha estiverem incorretos, o sistema exibe a mensagem de erro e mantém o jogador na mesma tela.
Pós-condição	Sessão do jogador ativa. No fluxo A, usuário gravado na tabela usuario.

UC02 – Iniciar partida	
Ator principal	Jogador autenticado.
Ator secundário	Banco de Dados MySQL.
Requisitos relacionados	RFS03, RFS04, RNF05.
Pré-condição	O jogador está autenticado (UC01).
Fluxo principal	1. Na tela inicial, o jogador escolhe “Novo Jogo”. 2. O sistema cria a partida com data de início, status “Em andamento”, vida inicial e pontuação zero. 3. O sistema associa a partida à primeira ilha (menor ordem). 4. O sistema exibe a ilha, a tartaruga, a vida e a pontuação.
Fluxo alternativo A – Continuar	Se houver partida em andamento, o sistema oferece continuar ou começar outra. Ao escolher “Continuar”, o sistema retoma a ilha, a vida e a pontuação salvas.
Fluxo alternativo B – Falha	Se não for possível acessar o banco de dados, o sistema exibe uma mensagem de erro e permanece na tela inicial, sem encerrar abruptamente.
Pós-condição	Partida em andamento registrada no banco de dados, com a tartaruga posicionada em uma ilha.

UC03 – Responder ao quiz da ilha	
Ator principal	Jogador.
Ator secundário	Banco de Dados MySQL.
Requisitos relacionados	RFS05, RFS06, RNF02, RNF03.
Pré-condição	Há uma partida em andamento e o quiz da ilha atual foi liberado (inimigo derrotado, RFS05).
Fluxo principal	1. O sistema seleciona uma pergunta da ilha ainda não respondida na partida e exibe o enunciado e as alternativas. 2. O jogador escolhe uma alternativa. 3. O sistema registra a escolha vinculada à partida. 4. O sistema confere no servidor se a alternativa é a correta. 5. O sistema exibe o resultado e a explicação e atualiza acertos e pontuação. 6. Os passos 1 a 5 se repetem até o jogador atingir os acertos necessários da ilha. 7. O sistema marca a ilha como concluída e libera a próxima (RFS04).
Fluxo alternativo A – Resposta errada	O sistema não soma pontos, exibe a explicação e apresenta a próxima pergunta.
Fluxo alternativo B – Sem perguntas	Se não houver perguntas disponíveis na ilha, o sistema exibe uma mensagem informando o problema, sem encerrar abruptamente.
Pós-condição	Respostas registradas, pontuação atualizada e, quando aplicável, ilha concluída.

4.2 DIAGRAMA DE CLASSE

O diagrama de classes representa as principais entidades do sistema e seus relacionamentos. As classes de domínio correspondem às tabelas do banco de dados (modelagem física da pasta BD), com métodos que representam as regras do jogo. Também fazem parte do modelo as enumerações StatusPartida (EmAndamento, Concluida, Derrotada), Dificuldade (Facil, Media, Dificil) e TipoPergunta (MultiplaEscolha, VerdadeiroFalso).

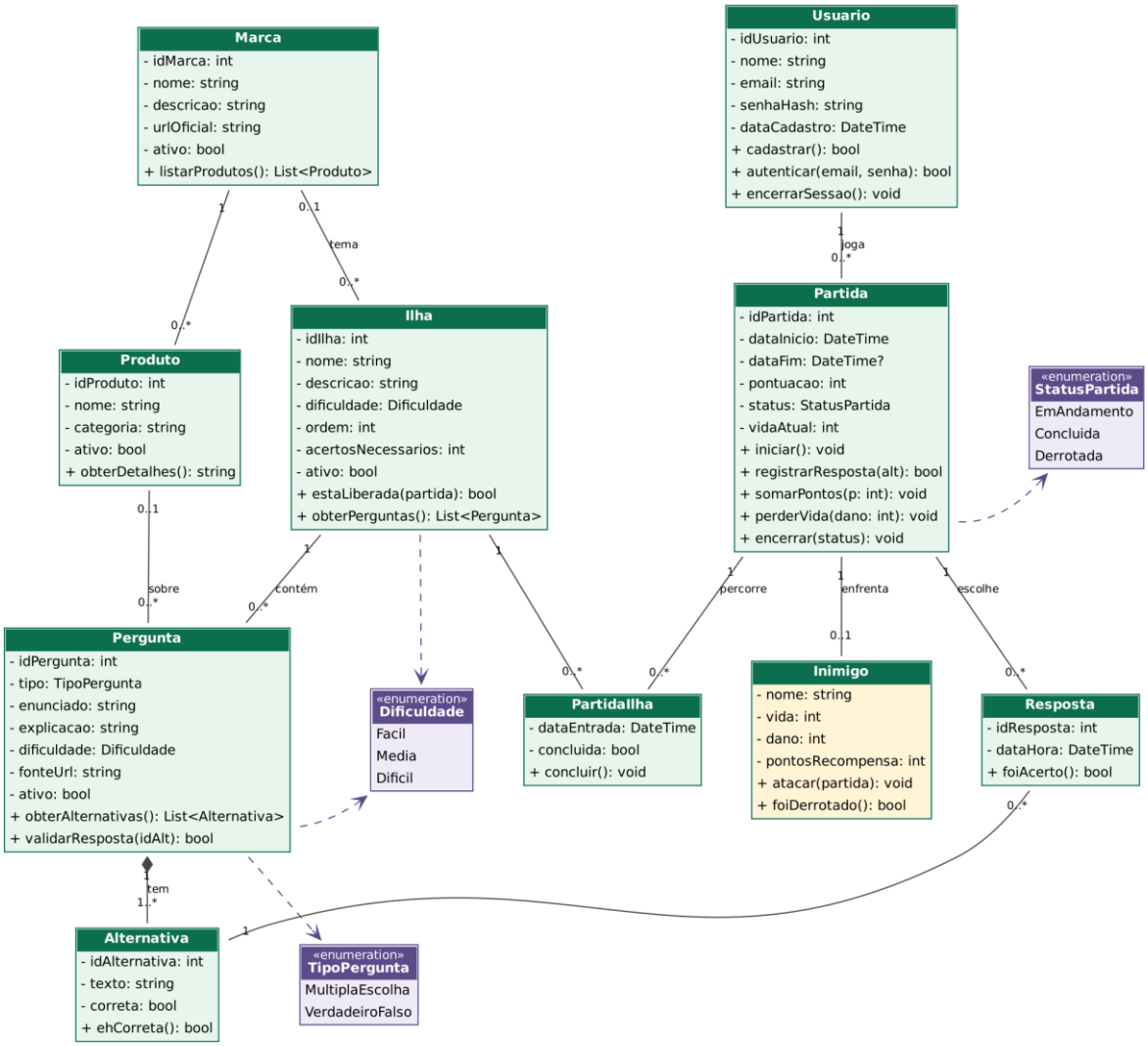


Figura 4: Diagrama de classes do ARCOR – Desafio das Marcas.

Relacionamentos principais:

- Usuario 1 — 0..* Partida: um usuário joga várias partidas e cada partida pertence a um único usuário.
- Partida 0..* — 0..* Ilha, por meio de PartidaIlha: registra por quais ilhas a tartaruga passou e se cada uma foi concluída.
- Partida 0..* — 0..* Alternativa, por meio de Resposta: registra as respostas dadas durante a partida.

- Ilha 1 — 0..* Pergunta e Pergunta 1 — 1..* Alternativa: a pergunta pertence a uma ilha e é composta por suas alternativas.
- Marca 1 — 0..* Produto, Marca 0..1 — 0..* Ilha e Produto 0..1 — 0..* Pergunta: ligam o conteúdo do quiz às marcas e aos produtos da Arcor.

A classe Inimigo representa o combate das ilhas (RFS05). Ela é uma classe de regra de jogo, criada em memória de acordo com a dificuldade da ilha, e não possui tabela correspondente no banco de dados atual.

5. ESPECIFICAÇÃO DA ESTRUTURA DE UMA ROTINA DO SISTEMA

5.1 Especificação da rotina

Rotina: Registrar resposta do quiz (ResponderPergunta)	
Sistema/Software	ARCOR – Desafio das Marcas (Jogo da Tartaruga).
Função	Registrar e validar a resposta do jogador a uma pergunta do quiz.
Descrição	Recebe a alternativa escolhida pelo jogador, confere no servidor se ela é a correta, registra a resposta na partida, calcula os pontos conforme a dificuldade da pergunta e verifica se a ilha foi concluída.
Entradas	idPartida, idPergunta e idAlternativa.
Fonte	Jogador (tela do quiz). Perguntas, alternativas e parâmetros de pontuação lidos do banco de dados MySQL.
Saídas	Se acertou (verdadeiro/falso), explicação da pergunta, pontos ganhos, pontuação atualizada e se a ilha foi concluída.
Destino	Tela do jogador (resultado e explicação) e tabelas resposta, partida e partida_ilha do banco de dados.
Ação	1. Verificar se a partida existe e está em andamento. 2. Verificar se a pergunta ainda não foi respondida nesta partida. 3. Verificar se a alternativa pertence à pergunta. 4. Consultar se a alternativa é correta e, se for, buscar os pontos da dificuldade na tabela parametro. 5. Gravar a resposta e somar os pontos à partida em uma única transação. 6. Contar os acertos do jogador na ilha e comparar com os acertos necessários. 7. Se atingiu, marcar a ilha como concluída em partida_ilha. 8. Devolver o resultado ao jogador.
Requisitos	RFS06, RNF02, RNF03 e RNF05.
Pré-condição	Jogador autenticado, partida em andamento e pergunta pertencente à ilha atual.
Pós-condições	Resposta registrada, pontuação atualizada e, quando os acertos necessários são atingidos, ilha marcada como concluída.
Efeitos colaterais	A pergunta respondida não é mais sorteada nesta partida. A conclusão da ilha libera a próxima (RFS04).

5.2 Codificação da rotina

Codificação em C#, com acesso ao banco por consultas parametrizadas nos repositórios:

```
public ResultadoResposta ResponderPergunta(int idPartida, int idPergunta, int idAlternativa)
{
    Partida partida = _partidas.Obter(idPartida);
    if (partida == null || partida.Status != StatusPartida.EmAndamento)
        throw new InvalidOperationException("Partida inválida ou encerrada.");

    if (_respostas.JaRespondida(idPartida, idPergunta))
        throw new InvalidOperationException("Pergunta já respondida nesta partida.");

    Alternativa escolhida = _alternativas.Obter(idAlternativa, idPergunta);
    if (escolhida == null)
        throw new ArgumentException("A alternativa não pertence à pergunta.");

    Pergunta pergunta = _perguntas.Obter(idPergunta);
    bool acertou = escolhida.Correta;
    int pontos = acertou ? _parametros.PontosPor(pergunta.Dificuldade) : 0;

    using (var transacao = _bd.IniciarTransacao())
    {
        _respostas.Registrar(idPartida, idPergunta, idAlternativa);
        if (acertou) _partidas.SomarPontos(idPartida, pontos);
        transacao.Confirmar();
    }

    Ilha ilha = _ilhas.Obter(pergunta.IdIlha);
    int acertosNaIlha = _respostas.ContarAcertos(idPartida, ilha.IdIlha);
    bool ilhaConcluida = acertosNaIlha >= ilha.AcertosNecessarios;
    if (ilhaConcluida) _partidas.ConcluirIlha(idPartida, ilha.IdIlha);

    return new ResultadoResposta(acertou, pergunta.Explicacao, pontos, ilhaConcluida);
}
```

6. ARQUITETURA DO SISTEMA

6.1 Visão Geral do Sistema

- **Nome do Sistema:** ARCOR – Desafio das Marcas (Jogo da Tartaruga).
- **Objetivo do Sistema:** promover a familiarização do jogador com a Arcor, suas marcas, linhas e produtos por meio de uma aventura em que uma tartaruga viaja de ilha em ilha e responde a perguntas de um quiz armazenadas em banco de dados.
- **Público-Alvo:** público juvenil.
- **Principais Funcionalidades:** cadastro e login de usuários; início e continuação de partidas; percurso por ilhas com dificuldade progressiva; combate contra inimigos; quiz com explicação educativa após cada resposta; pontuação e progresso salvos no banco de dados.

6.2 Arquitetura Geral

O sistema adota a Arquitetura em Camadas (Layered Architecture), com separação clara entre apresentação, aplicação, regras de negócio (domínio) e persistência. O frontend se comunica com o backend por uma API REST, trocando mensagens em formato JSON sobre HTTP, e somente a camada de persistência acessa o banco de dados. Essa separação atende ao requisito de manutenibilidade

(RNF06): é possível ampliar o conteúdo do jogo apenas inserindo registros no banco, sem alterar a lógica.

6.3 Componentes Arquiteturais e Camadas

6.3.1 Camada de Apresentação (Frontend)

- **Tecnologias:** HTML5, CSS3 e JavaScript.
- **Responsabilidades:** interface com o usuário (telas de login, mapa de ilhas, combate e quiz); validação inicial dos dados; comunicação com a API via HTTP/JSON; layout responsivo.

6.3.2 Camada de Aplicação (Backend/API)

- **Tecnologias:** C# com ASP.NET Core (Web API).
- **Responsabilidades:** receber as requisições do frontend; autenticação e autorização por sessão; chamar os serviços de domínio; exposição de endpoints RESTful (por exemplo, cadastro, login, partidas e respostas).

6.3.3 Camada de Domínio (Negócio)

- **Descrição:** contém as regras de negócio e os modelos de domínio, independentes das demais camadas.
- **Componentes:** entidades (Usuario, Partida, Ilha, Pergunta, Alternativa, Resposta, Marca, Produto, Inimigo); casos de uso (autenticar, iniciar partida, responder ao quiz); regras de validação (pontuação por dificuldade, acertos necessários para liberar a próxima ilha, fim da partida quando a vida chega a zero).

6.3.4 Camada de Persistência (Banco de Dados)

- **Tecnologias:** MySQL 8 e a biblioteca MySqlConnector, com consultas parametrizadas.
- **Responsabilidades:** armazenamento e recuperação de dados (usuários, partidas, ilhas, marcas, produtos, perguntas, alternativas e respostas); scripts de criação e carga do banco (BD/schema.sql).

6.4 Diagrama da Arquitetura

O diagrama mostra o fluxo entre navegador, camadas do servidor web e banco de dados, além dos componentes de infraestrutura previstos na topologia de rede do projeto (Cloudflare, balanceador de carga e cache Redis). A chamada a um servidor externo em JSON é a integração desenvolvida na disciplina de Redes de Computadores.

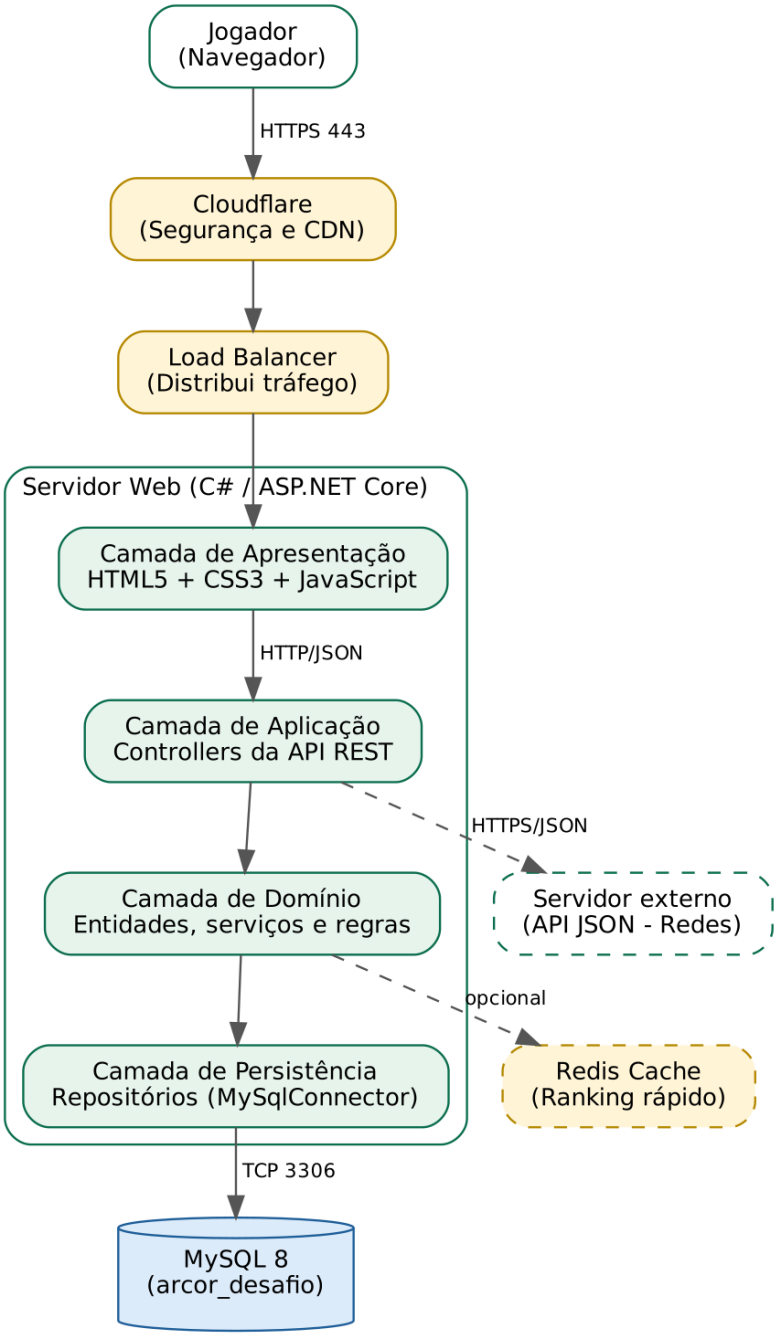


Figura 5: Diagrama da arquitetura do ARCOR – Desafio das Marcas.

Exemplo textual do fluxo:

```
[Jogador / Navegador]
        |  HTTPS
[Cloudflare] -> [Load Balancer]
        |
[Frontend: HTML5 + CSS3 + JavaScript]
        |  HTTP/JSON
[ASP.NET Core (API REST)] -> [Domínio: regras do jogo]
        |  SQL parametrizado (porta 3306)
[MySQL 8 - banco arcor_desafio]
```

6.5 Organização de Pastas e Módulos

O código é organizado no repositório do GitHub conforme a estrutura abaixo, que segue as pastas exigidas nas entregas das disciplinas:

```
/ (repositório do projeto)
├── BD/
│   ├── README.md        (modelagem física documentada)
│   └── schema.sql       (criação das tabelas e carga inicial)
├── SRC/
│   ├── Arcor.Api/       (controllers, Program.cs, appsettings.json)
│   ├── Arcor.Dominio/   (entidades, serviços e regras de negócio)
│   ├── Arcor.Dados/     (repositórios e acesso ao MySQL)
│   └── Arcor.Web/       (HTML, CSS, JavaScript e imagens)
├── Algoritmos e Lógica de Programação/
└── DOCUMENTOS/          (relatórios, diagramas e banner)
```

6.6 Integrações Externas

- Servidor externo com resposta em JSON: chamada HTTP desenvolvida na disciplina de Redes de Computadores (pasta SRC).
- Cloudflare (segurança e CDN): componente previsto na topologia de rede do projeto.
- O sistema não utiliza APIs de pagamento, mapas ou redes sociais, e a autenticação é feita pelo próprio sistema (email e senha).

6.7 Ambiente e Implantação

- **Ambientes:** desenvolvimento (computador da equipe, com MySQL local) e apresentação na Semana FECAP de Tecnologia. Homologação e produção ficam como evolução do projeto.
- **Contêineres:** Docker/Docker Compose pode ser usado para subir a aplicação e o MySQL de forma padronizada (opcional).
- **CI/CD:** versionamento e colaboração pelo GitHub; GitHub Actions pode automatizar a compilação (opcional).
- **Hospedagem:** não definida nesta etapa; para a demonstração, o sistema roda no computador da equipe.

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. Engenharia de Software. 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.

ARCOR BRASIL. Site institucional. Disponível em: https://www.arcor.com.br/. Acesso em: nov. 2026.

ARCOR BRASIL. Marcas Tortuguita e Triunfo. Disponível em: https://www.arcor.com.br/marcas/. Acesso em: nov. 2026.

FECAP. Projeto Interdisciplinar – ARCOR Desafio das Marcas. 1º ADS, 2º semestre de 2026.